
DNS++: A Privacy-First, Dynamic, Location-Aware Name Resolution System — A Real-System Implementation and Empirical Evaluation

Anonymous authors

Paper under double-blind review

Abstract

The Domain Name System exposes query intent to intermediaries, cannot keep pace with dynamic service changes, and lacks location-aware replica selection. The DNS++ proposal addresses all three through a hierarchical publish/subscribe overlay with homomorphic encryption, but has only been evaluated in simulation. We present the first real-system implementation of the DNS++ broker in C++17 — a TLV binary protocol, an epoll event loop, Algorithm 1 (Proximity Routing) with a quadrant-based propagation brake, a multi-broker overlay with dynamic Minimum Bounding Hyperrectangle aggregation, and the modified Paillier privacy layer — and evaluate it over 30 independent trials per configuration with bootstrap confidence intervals. Three findings follow. The propagation brake is far blunter than simulation suggests: in a three-broker tree it buys a 6.9% reduction in forwarding events while destroying 77% of correct nearest-replica resolutions, whereas the quadrant cache alone attains a traffic ratio of 1.168 at 0.981 recall. The privacy layer’s broker-side cost is bounded and negligible: a homomorphic Match costs 0.0127 ms against 23.5–59 ms for endpoint blinding, bounding broker overhead at $S \times 0.0127$ ms in the number of subscription groups S ; a paired comparison over 30 trials finds no detectable latency difference ($p = 0.299$). Convergence after replica migration is directional: when a replica moves closer, subscribers converge in a median of 0.42 ms, five orders of magnitude faster than a DNS TTL; when a replica moves away, the push-only monotone filter provides no convergence at all (0 of 1391 cases). We report this asymmetry, and the plaintext exposure introduced by hash-based subscription grouping, as the principal limitations of the design as implemented.

1 Introduction

Every Internet interaction begins with a name resolution. The Domain Name System handles billions of these lookups daily and underpins every other Internet service, but it was designed for a much simpler era, and three structural limitations have become critical. It **leaks privacy**: every lookup reveals who is asking for what, and when, to the intermediaries handling it (Wicinski, 2021). It **cannot keep up with change**: services scale, fail and move constantly, yet DNS updates lag minutes to hours (Mockapetris, 1987). And it **lacks location awareness**: nothing in the protocol steers a user to a nearby replica.

Existing solutions address one crack each. Encrypted transports — DoT (Hu & Zhu, 2016), DoH (Hoffman & McManus, 2018), DoQ (Huitema et al., 2022) — hide query content from observers but still expose client identity to resolvers. Oblivious DNS (Schmitt et al., 2019) unlinks identity from query but remains pull-based, with no dynamic updates or locality. Private Information Retrieval (Chor et al., 1998; Bhat & Sunitha, 2019) offers strong theoretical privacy at prohibitive cost, and again addresses neither dynamics nor locality. Deployed DNS obtains locality through EDNS Client Subnet (Contavalli et al., 2016), which improves replica selection *precisely by* disclosing a prefix of the client’s address to every server on the resolution path — trading away the privacy the encrypted transports were introduced to protect. That tension is the gap DNS++ (Rio et al., 2026) targets: it promises all three properties in one system, through a hierarchical pub/sub overlay, homomorphic matching, and multi-dimensional spatial routing. It has, however, only ever been simulated.

This work presents the first real-system implementation, in C++17, and measures it. Our contributions are fourfold. We deliver a working broker together with a reproducible harness in which every experiment is seeded per trial, warmed up, and accompanied by machine-recorded environment metadata and kernel

packet-loss counters. We correct the characterisation of the propagation brake: it is not a tunable efficiency knob but an overload valve, because the quadrant cache — which suppresses selectively rather than by rate — already does the useful filtering. We replace the usual undetectable-difference argument about homomorphic overhead with a *bound*: broker-side cost is $S \times 0.0127$ ms in the number of subscription groups, which both explains why no difference is measurable and predicts how the cost scales. And we provide the first measurement of DNS++ convergence under replica migration, which yields a negative result we treat as a finding rather than an anomaly.

We are equally explicit about what the implementation does not achieve. To restore subscriber grouping under semantically secure ciphertexts, our brokers index routing state by a hash of the plaintext service name, which the client already transmits; the broker therefore learns the name, and the homomorphic Match becomes a formal re-check rather than the mechanism deciding routing. Section 6.4 states this and the remaining deviations in full.

2 Background and Related Work

DNS privacy. DoT (Hu & Zhu, 2016), DoH (Hoffman & McManus, 2018) and DoQ (Huitema et al., 2022) encrypt queries in transit, preventing on-path observation but not resolver-side observation of both client identifier and queried name; they also reduce operator visibility, raising monitoring concerns (Hynek, 2023). ODoH (Schmitt et al., 2019) interposes a proxy so that no single entity links user to query, but stays a pull-based query-response model with no support for dynamic updates or replica selection. PIR-based resolution (Chor et al., 1998; Bhat & Sunitha, 2019; Papadopoulos et al., 2020) lets clients retrieve records without revealing which, at computational and bandwidth costs that impede deployment, and likewise ignores dynamics and locality.

Location awareness in deployed DNS. Production systems obtain locality by two retrofits, both in tension with the above. EDNS Client Subnet (Contavalli et al., 2016) attaches a truncated client prefix to the query so authoritative servers can select a nearby replica — effective, widely deployed, and directly opposed to the threat model DoH and ODoH address. The alternative moves selection out of DNS into IP anycast and CDN request routing, where catchments follow BGP policy rather than geography and frequently deviate from true proximity (Calder et al., 2015). Both share a structural property: selection is decided by infrastructure that must learn something about the client, and neither offers a push path for service changes.

Directory services and spatial pub/sub. X.500 (Chadwick, 1994) established hierarchical directories with strong naming semantics but heavyweight protocols, motivating LDAP (Sermersheim, 2006); DHT-based discovery (Ratnasamy et al., 2001; Kurte et al., 2024) scales but offers neither privacy nor locality; Agent Name Service (Huang et al., 2025) addresses agent discovery without privacy-preserving matching. Structurally, DNS++ belongs to the spatial pub/sub lineage: Meghdoot (Gupta et al., 2004) maps subscriptions and events onto a CAN overlay so matching becomes a spatial lookup, and Scribe (Castro et al., 2002) builds locality-aware multicast trees over Pastry. Our quadrant filter is of the same family — a parent retains, per child and quadrant, the best distance forwarded so far. What distinguishes DNS++ is not spatial routing but the requirement that the broker match *without learning the subscription*.

Privacy-preserving pub/sub. Nabeel et al. (2012) proposed a modified Paillier cryptosystem enabling brokers to route without learning content; Nabeel et al. (2013) added self-blinding, removing per-query interaction with a central authority. DNS++ adapts this to a hierarchical overlay with spatial routing. DP5 (Borisov et al., 2015) addresses private presence but not name resolution or replica selection.

The gap. No existing class occupies all three requirements simultaneously (Table 1). Encrypted transports give confidentiality without dynamics or locality; ECS gives locality at the direct expense of privacy; spatial pub/sub gives locality and push-based dynamics but assumes a trusted broker; privacy-preserving pub/sub gives confidential matching but is not spatial. DNS++ claims all three and has never been built. This work fills that gap, and establishes which claims survive contact with an implementation.

Table 1: Coverage of the three requirements. • = addressed, ◦ = partially addressed, blank = not addressed.

System class	Privacy	Dynamics	Locality
DoT / DoH / DoQ (Hu & Zhu, 2016; Hoffman & McManus, 2018; Huitema et al., 2022)	◦		
ODoH (Schmitt et al., 2019)	•		
PIR-based DNS (Bhat & Sunitha, 2019; Papadopoulos et al., 2020)	•		
EDNS Client Subnet (Contavalli et al., 2016)			•
Anycast / CDN routing (Calder et al., 2015)		◦	◦
Spatial pub/sub (Gupta et al., 2004; Castro et al., 2002)		•	•
Privacy-preserving pub/sub (Nabeel et al., 2012)	•	•	
DNS++ (Rio et al., 2026)	•	•	•

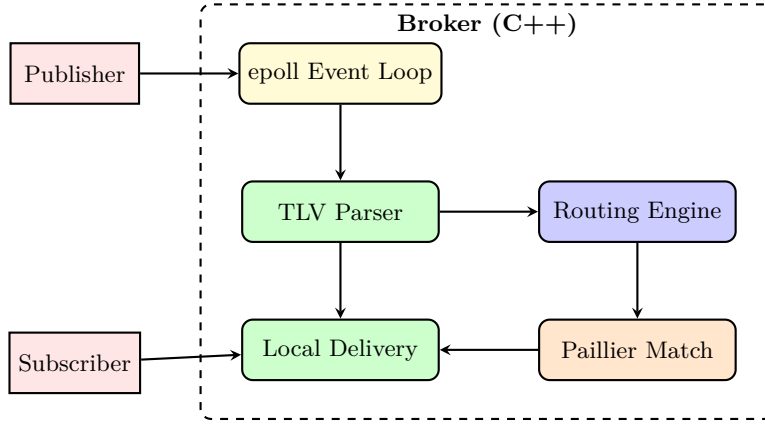


Figure 1: DNS++ broker architecture. The epoll loop reads UDP datagrams, the zero-copy parser decodes TLV fields, and the routing engine consults the IT[]/OT[] tables. The Match engine intercepts encrypted publications before local delivery.

3 System Design

3.1 Architecture and Protocol

Broker processes form a tree mediating between publishers (service replicas announcing availability) and subscribers (clients resolving a name to a locator). Each broker runs a level-triggered epoll loop over a single UDP socket, parses a custom Type–Length–Value binary protocol, and maintains per-service routing tables implementing Algorithm 1. Figure 1 shows the internal structure.

The wire format is an 8-byte fixed header (message type, TLV count, payload length) followed by variable TLV fields and an optional payload; each field carries a 2-byte type, 2-byte length and variable value. This admitted the later addition of blinded values and an extended statistics record as new types without structural change. Integers use network byte order; floats are byte-swapped through a 32-bit integer reinterpretation. Appendix A lists the fields.

3.2 Proximity Routing

Algorithm 1 states the publication path as implemented. A subscriber’s SUBSCRIBE inserts it into the input table with a `cached_closest_dist` initialised to infinity; a `QUERY_MODE` flag additionally returns the nearest cached publication immediately, giving zero-latency discovery of existing services. A publication is cached unconditionally — so query-mode answers survive later suppression — and then passes the upward brake, the per-child quadrant cache, the local brake, and finally the per-subscriber monotone filter. That last filter is what gives “Closest” semantics: a subscriber is told about a replica only if it is strictly nearer

Algorithm 1 Proximity Routing — publication handling at a broker

Require: publication p at (lat_p, lon_p) ; routing key k ; configuration $brakeScope$

```
1: CACHEPUBLICATION( $k, p$ ) ▷ unconditional: query-mode depends on it
2: if  $hasParent$  and  $p$  not from parent and  $brakeScope \in \{upward, both\}$  then
3:   if BRAKEALLOWS( $k, lat_p, lon_p$ ) then forward  $p$  upward else  $braked_{up}++$ 
4: end if
5: for all child brokers  $c$  active for  $k$  do
6:    $q \leftarrow \text{QUADRANTOF}(c.region, lat_p, lon_p)$ ;  $d \leftarrow \text{dist}(p, \text{QUADRANTCENTRE}(c, q))$ 
7:   if  $d < c.closestQuad[q]$  then ▷ information-aware filter
8:      $c.closestQuad[q] \leftarrow d$ ; forward  $p$  down to  $c$ 
9:   end if
10: end for
11: if  $brakeScope \in \{local, both\}$  and not BRAKEALLOWS( $k, lat_p, lon_p$ ) then
12:    $braked_{local}++$ ; return
13: end if
14: for all subscribers  $s$  in group  $k$  do
15:   if  $\text{dist}(p, s) < s.cachedClosestDist$  then deliver  $p$  to  $s$ ;  $s.cachedClosestDist \leftarrow \text{dist}(p, s)$ 
16: end for
```

than anything it already knows. Section 5.6 shows it is also the source of the system’s principal dynamic limitation.

Propagation brake. The brake limits how many publications a broker propagates from one spatial quadrant within a sliding window. Each quadrant holds a timestamp queue; a publication propagates only if fewer than `brake_limit` entries fall inside `brake_window` seconds. A `brake_scope` parameter (`upward`, `local`, `both`; default `both`) selects which directions are gated. The distinction is material: under `upward` the brake has no effect at all on a single broker, which has no parent. We note this because an earlier revision of this implementation gated only the upward path, silently rendering our own single-broker brake sweep vacuous.

Distance. We use the equirectangular approximation $d = \sqrt{\Delta lat^2 + (\Delta lon \cos lat)^2}$, sufficient for *ranking* candidates and avoiding Haversine’s trigonometric cost on the hot path. Section 6.4 notes where it departs from true distance.

3.3 Hierarchical Overlay

Each broker reads identifier, listen port, parent address, coordinates and brake parameters from a configuration file (Appendix B). Children send HELLO on startup, registering coordinates; the parent computes its Minimum Bounding Hyperrectangle as the union of its children’s regions. A leaf receiving a local subscription propagates it upward, and the parent records which children are active for that name. Publications then flow in three directions: upward subject to the brake, downward subject to quadrant filtering, and locally subject to the brake and then the monotone filter. Leaves update their MBH as clients register and propagate changes upward via REGION_UPDATE, so parents make quadrant decisions against accurate coverage.

3.4 Privacy-Preserving Matching

Subscriptions and publications carry blinded values rather than names. We implement the modified Paillier cryptosystem of (Rio et al., 2026), following the protocols of Nabeel et al. (2012). A Homomorphic Encryption Parameter Service (HEPS) generates a 2048-bit key pair; the private key λ blinds, while (n, g, μ) suffices for the unblinding brokers perform. Three mechanisms underpin the protocol. *Blinding cancellation:* the HEPS issues (e_m, d_m) with $d_m = n - e_m$, and since $g = 1 + n$ has order exactly n in $(Z/n^2Z)^*$, $g^{e_m} g^{d_m} = g^n \equiv 1$. *Randomness stripping:* raising a ciphertext to the λ -th power eliminates its randomness, as $r^{n\lambda} \equiv 1 \pmod{n^2}$ by Carmichael’s theorem. *Value-domain separation:* a fixed stretching factor r_m and per-execution random r place positive differences in $[0, n/2)$ and negative ones in $(n/2, n)$.

Lemma 1. Let $bval_n(x) = g^{e_m} g^{\lambda(r_m x + r)} \bmod n^2$ and $bval_m(-v) = g^{d_m} E(-v)^{r_m \lambda} \bmod n^2$, with $e_m + d_m \equiv 0 \pmod{\text{ord}(g)}$ and $g = 1 + n$. Then $L(bval_n(x) \cdot bval_m(-v) \bmod n^2) \cdot \mu \equiv r_m(x - v) + r \pmod{n}$.

Corollary 1 (Correctness of the threshold test). With $0 < r < r_m$ and $r_m|x - v|_{\max} + r < n/2$, we have $\text{diff} < n/2 \iff x \geq v$.

Proof of Lemma 1. We treat the three factors in turn.

(i) *Randomness cancellation.* A Paillier ciphertext is $E(-v) = g^{-v} r_e^n$ for random r_e . Raising to the $r_m \lambda$ -th power gives $E(-v)^{r_m \lambda} = g^{-v r_m \lambda} \cdot r_e^{n r_m \lambda}$. The group $(\mathbb{Z}/n^2\mathbb{Z})^*$ has exponent $n\lambda$, so $r_e^{n\lambda} \equiv 1 \pmod{n^2}$ by Carmichael’s theorem, hence $r_e^{n r_m \lambda} \equiv 1$. The encryption randomness is eliminated exactly, not merely masked.

(ii) *Blinding cancellation.* Multiplying the two blinded values, the blinding factors contribute $g^{e_m} \cdot g^{d_m} = g^{e_m + d_m} \equiv 1 \pmod{n^2}$ by assumption.

(iii) *Recovery.* What remains is $g^{\lambda k}$ with $k = r_m(x - v) + r$. Using $g = 1 + n$, binomial expansion modulo n^2 gives $(1 + n)^{\lambda k} \equiv 1 + \lambda k n$. Therefore $L(g^{\lambda k}) = ((1 + \lambda k n) - 1)/n = \lambda k$, and multiplying by $\mu \equiv \lambda^{-1} \pmod{n}$ yields k . $\square$

Proof of Corollary 1. If $x = v$ then $\text{diff} = r < r_m < n/2$. If $x > v$ then $\text{diff} \geq r_m$, and remains below $n/2$ by the parameter constraint. If $x < v$ then $r_m(x - v) + r$ is negative and, reduced modulo n , lands in $(n/2, n)$. $\square$

Equality is decided by two invocations — verifying $x \geq v$ against $bval_m(-v)$ and $x < v + 1$ against $bval_m(-(v + 1))$ — and a match declared only if both hold. The broker learns the two Boolean outcomes; note, however, that the recovered magnitude diff encodes $r_m(x - v) + r$, so a party holding r_m can extract ordering information about hashed names even without λ .

Subscription grouping and its cost. Blinded values are semantically secure: the same name yields a different ciphertext each time. A ciphertext therefore cannot serve as a lookup key — indexing on $bval$ would place every subscriber in a group of one, forcing a Match per subscriber per publication and destroying the sharing that keeps broker cost proportional to *groups*. Our implementation instead indexes on a hash of the plaintext service name, which the client already transmits alongside the blinded values. This restores grouping and yields the broker-side costs of Section 5.5. It also means the broker learns the name, and the homomorphic Match becomes a formal re-check rather than the mechanism deciding routing. We record this as a deliberate trade-off and quantify it in Section 6.4; the Cover protocol of Nabeel et al. (2012), which aggregates subscriptions without revealing them, is the principled alternative and remains future work.

4 Implementation

The system is C++17 built with CMake. Level-triggered epoll suits datagram semantics — each `recvfrom()` returns one complete message and readiness persists while data remains — and the epoll timeout doubles as the TTL garbage-collection timer, avoiding extra descriptors. Parsing is zero-copy: the buffer is reinterpreted as the fixed header and traversed by pointer arithmetic with length validation before each access. Socket descriptors follow RAII, and a dedicated logger thread drains a producer–consumer queue so that log I/O never blocks the event loop.

The Paillier layer is built over GMP. Key generation forces the top two bits of each prime factor, guaranteeing a modulus of exactly the requested width; without this, $p \cdot q$ can fall short, and we observed moduli between 2040 and 2048 bits before the fix. The random state is seeded once per instance rather than per operation, the previous per-call reseeding having invoked `std::random_device` on every encryption and materially inflated measured cryptographic cost. Expensive exponentiations are confined to publishers and subscribers; brokers execute only modular multiplications and $L(\cdot)$.

Observability. Because a silent fallback to plaintext would be indistinguishable from a successful encrypted run, brokers export twelve counters (`STATS_DATA_EXT`), including Match invocations and hits, publications suppressed in each direction, subscription-group count, and whether homomorphic matching is active. Every

Table 2: Single-broker brake sweep (10 publishers, 50 subscribers, 30 trials). Suppression is *braked_local/pub_received*. Brackets are 95% bootstrap CIs.¹

Brake limit	Recall	Stretch	Suppression
1	0.452 [0.404, 0.499]	2.172 [1.970, 2.384]	0.624
2	0.749 [0.699, 0.797]	1.373 [1.262, 1.496]	0.321
4	0.983 [0.965, 0.996]	1.013 [1.001, 1.030]	0.033
1000	1.000 [1.000, 1.000]	1.000 [1.000, 1.000]	0.000

encrypted result in Section 5 is accompanied by a non-zero Match count. Key loading fails fast unless a flag explicitly permits plaintext fallback, and any fallback is announced at startup. We assume commodity Linux x86_64, co-located processes with no emulated wide-area latency, two-dimensional coordinates, and benchmark clients in place of real applications.

5 Evaluation

5.1 Setup

Experiments ran on Ubuntu Linux, kernel 7.0.0-30-generic, on an AMD Ryzen 9 8945HS with two cores available to the virtual machine, GMP 6.3.0, GCC 13.3.0, CMake **Release**. Publishers and subscribers are benchmark clients placed uniformly at random over latitude $[-90, 90]$ and longitude $[-180, 180]$. We measure **recall** (proportion of subscribers receiving their true nearest publisher), **stretch** (received distance divided by ground-truth nearest, optimal 1.0), **traffic ratio** (total forwarding events over successful local deliveries), and **convergence time** after a migration. An effectively unbounded brake serves as the internal baseline; we implement it as `brake_limit=1000`, at which the measured suppression rate is exactly zero, and label it ∞ in figures. Plaintext matching is the baseline for the cryptographic evaluation.

Each configuration was repeated for 30 independent trials, each with its own derived seed, so trials are independent samples rather than repetitions of one placement. Three warm-up trials precede every measured run and are discarded. We report means with 95% bootstrap confidence intervals (10,000 resamples, percentile method), appropriate here because recall and stretch are bounded and symmetric standard-deviation bars would extend past those bounds. The plaintext-encrypted comparison uses a paired Wilcoxon signed-rank test.

5.2 Single-Broker Brake Sweep

We sweep `brake_limit` over $\{1, 2, 4, 1000\}$ with 10 publishers and 50 subscribers and `brake_scope=both`; with no parent, only the local arm is active. Table 2 reports recall and stretch with the *measured* suppression rate, read from the `braked_local` counter rather than inferred from recall.

The brake behaves as designed, and the trade-off is steep. At `brake_limit=1` the broker suppresses 62.4% of publications (206 of 330) and recall falls to 0.452, with surviving subscribers reaching replicas $2.17\times$ farther than optimal on average. At 4, suppression is 3.3% and recall recovers to 0.983 at near-optimal stretch. With the brake disabled, recall and stretch are exactly 1.000, confirming Algorithm 1 correct in the absence of throttling.

Recall consistently exceeds $1 - \text{suppression}$ — 0.452 against 0.376 at limit 1, 0.983 against 0.967 at limit 4. The gap is not noise: it reflects publications suppressed that would not have been anyone’s nearest replica. Suppression costs recall only when the suppressed publication was someone’s optimum, and under uniform placement most are not.

¹The broker’s counters are cumulative over process lifetime and therefore span 330 publications — the 300 of the measured trials plus the 30 of the three discarded warm-up trials — whereas recall and stretch are computed over measured trials only. Warm-up and measured trials draw from the same placement distribution, so the suppression rate is unaffected to three decimal places.

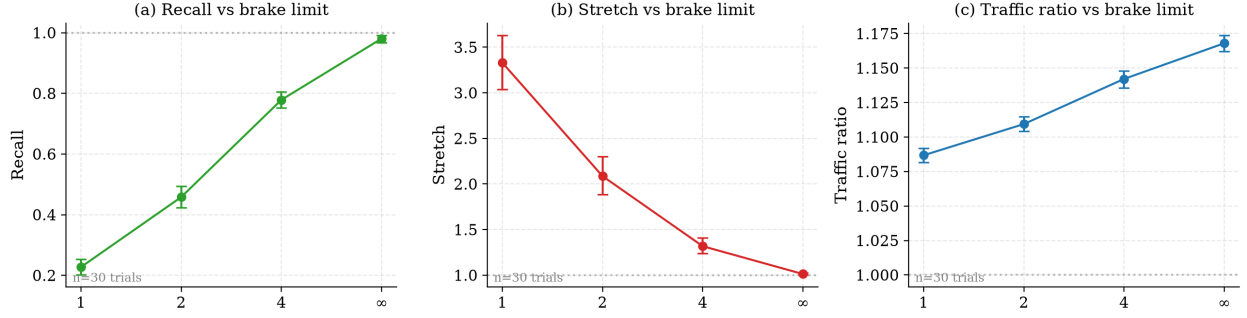


Figure 2: Multi-broker evaluation: recall, stretch and traffic ratio against brake limit (∞ denotes `brake_limit=1000`). Error bars are 95% bootstrap CIs over 30 trials. Three-broker tree, 20 publishers, 50 subscribers.

Table 3: Multi-broker brake sweep (three-broker tree, 20 publishers, 50 subscribers, 30 trials).

Brake limit	Recall	Stretch	Traffic ratio
1	0.227 [0.201, 0.253]	3.331 [3.036, 3.646]	1.087 [1.082, 1.092]
2	0.458 [0.424, 0.493]	2.086 [1.889, 2.299]	1.109 [1.104, 1.115]
4	0.779 [0.753, 0.805]	1.318 [1.236, 1.410]	1.142 [1.136, 1.148]
1000	0.981 [0.967, 0.992]	1.013 [1.003, 1.027]	1.168 [1.162, 1.174]

5.3 Multi-Broker Hierarchical Routing

We extend to a three-broker tree — one root, two leaves at London and Berlin — with 20 publishers and 50 subscribers attached to their nearest leaf, so both arms of the brake are active.

The brake is a poor bargain in a two-level tree. Tightening from 1000 to 1 reduces the traffic ratio from 1.168 to 1.087 — 6.9% fewer forwarding events — while recall collapses from 0.981 to 0.227 and stretch rises from 1.013 to 3.331. The most aggressive setting discards roughly three quarters of correct resolutions for a 7% traffic saving; even `brake_limit=4` costs 20 percentage points of recall for a 2.2% saving.

The quadrant cache does the useful filtering. With the brake disabled the traffic ratio is already only 1.168: each delivery costs 0.168 additional forwarding events. That floor is set by the downward quadrant cache, which forwards to a child only what improves its per-quadrant closest distance. The brake pushes a further 0.081 below this floor, but by discarding at a rate rather than by redundancy.

Braking compounds across levels. At the same limit, recall degrades further in the tree than in the single broker — 0.227 against 0.452 at limit 1 — because a publication must survive the leaf’s upward brake, the root’s forwarding decision, and the destination leaf’s local brake, and the suppression probabilities multiply. The counters confirm this: at limit 1, 17 publications were suppressed locally and 15 upward, a combined suppression exceeding one event per publication received.

Comparison with the original simulation. The qualitative trend agrees — recall falls and stretch rises as the brake tightens — but the magnitude does not: our brake is considerably more damaging at comparable settings. Two differences plausibly account for this. At our scale, 20 publishers over four global quadrants give roughly five publications per quadrant per window, so a limit of 1 or 2 binds almost immediately, whereas at simulation scale (10^6 publishers) the same limit is a far smaller fraction of offered load. And uniform placement produces no clustering, whereas population-weighted placement concentrates publishers where subscribers are, making a suppressed publication more likely to have a near-equivalent substitute. Both predict that an *absolute* brake count becomes progressively more aggressive as density falls — a design observation we return to in Section 6.1.

5.4 Scalability

Fixing `brake_limit=4` and 10 publishers, we vary subscribers from 10 to 1000, sampling kernel UDP statistics from `/proc/net/snmp` before and after each run. The 50-subscriber row therefore repeats the `brake_limit=4` configuration of Table 2, and is reproduced here for continuity.

Table 4: Subscriber-count scaling (`brake_limit=4`, 10 publishers, 30 trials). UDP errors are the measured delta in `RcvbufErrors`.

Subscribers	Recall	Stretch	UDP RcvbufErrors
10	0.983 [0.953, 1.000]	1.007 [1.000, 1.021]	0
50	0.983 [0.965, 0.996]	1.013 [1.001, 1.030]	0
200	0.982 [0.966, 0.995]	1.014 [1.004, 1.027]	0
500	0.879 [0.812, 0.938]	1.017 [1.005, 1.033]	1818
1000	0.833 [0.746, 0.905]	1.017 [1.005, 1.031]	4963

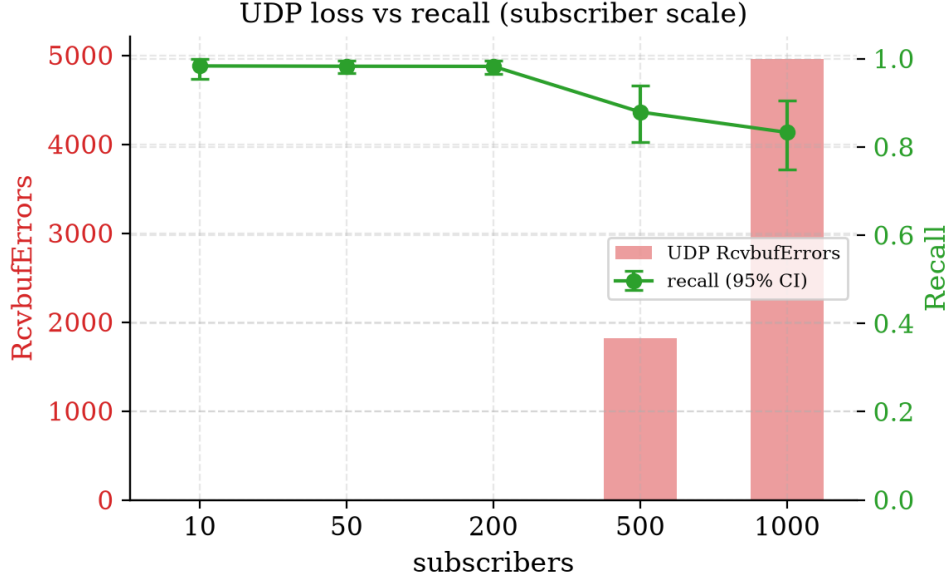


Figure 3: Kernel UDP receive-buffer overflows against subscriber count, overlaid with recall. Losses appear only beyond 200 subscribers and track the recall degradation.

Up to 200 subscribers recall holds at 0.98 with stretch within 1.5% of optimal and zero overflows. Beyond that, recall degrades to 0.879 and 0.833 while overflows rise to 1818 and 4963. The correlation does more than suggest a cause: the stretch column identifies the mechanism. *Stretch does not degrade* — it stays at 1.017 across the whole range, including where recall drops. Had routing been failing to identify nearest replicas, subscribers would have been steered to worse ones and stretch would have risen. Instead, subscribers that receive a publication still receive the optimal one; the losses are datagrams the kernel discarded before the client could read them, when hundreds of sockets are polled from one process on a two-core host. This is a limitation of the harness, not the algorithm, and precisely the class of effect a discrete-event simulation cannot exhibit. Enlarging `SO_RCVBUF` to 4 MiB (the kernel granted 8) did not eliminate it, indicating the bottleneck is the single-threaded polling loop rather than buffer capacity alone.

5.5 Privacy Layer Overhead

We measure the same workload (single broker, 10 publishers, 50 subscribers, `brake_limit=4`, 30 trials) in plaintext and encrypted modes with 2048-bit keys, with endpoint blinding performed before the timing window opens.

A paired Wilcoxon signed-rank test gives $W = 182$, $z = -1.04$, $p = 0.299$, median paired difference +0.38 ms. We cannot reject the hypothesis that the modes have equal per-publication latency.

The encrypted runs really were encrypted. This matters because a silent fallback would produce the same result. The counters report 319 `executeMatch` invocations in encrypted mode against 0 in plaintext;

Table 5: Plaintext versus encrypted matching (30 paired trials). See the caveat on absolute latency below.

Mode	Recall	Stretch	Latency mean (ms)	Median (ms)
Plaintext	0.983 [0.966, 0.996]	1.013 [1.001, 1.030]	63.6 [60.3, 67.1]	62.6
Encrypted	0.983 [0.966, 0.996]	1.013 [1.001, 1.030]	63.4 [60.6, 66.3]	63.0

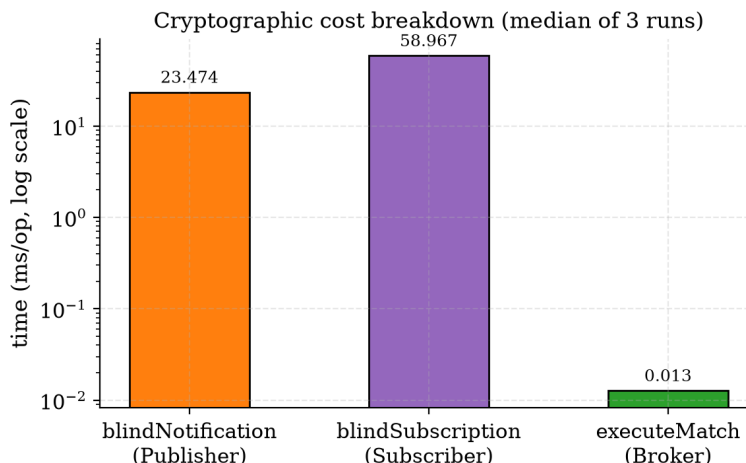


Figure 4: Cost decomposition of the privacy layer, logarithmic scale. Endpoint blinding dominates by more than three orders of magnitude.

recall and stretch are identical to three decimals, confirming that homomorphic Match reproduces plaintext routing decisions exactly.

We bound the broker-side cost rather than argue from a null result. Absence of a detectable difference is weak evidence alone. Appendix C measures a single Match at 0.0127 ms. A broker performs one Match per subscriber *group* per publication, so with S groups the added cost per publication is bounded by $S \times 0.0127$ ms. The group count reported by the counters is a snapshot taken after the run, once TTL expiry has retired inactive entries, and is therefore not a safe upper bound on S during the run; we instead take the worst case in which grouping fails entirely and each of the 50 subscribers forms its own group, giving $50 \times 0.0127 = 0.635$ ms. Even this pessimistic bound is under 1% of the measured per-publication latency and well below the trial-to-trial spread, so the end-to-end experiment could not have resolved it, and the bound explains why. It also extrapolates: cost grows linearly in *distinct subscription groups*, not in subscribers or publications, so 10^4 concurrent groups would add roughly 127 ms of broker CPU per publication.

The cost lives at the endpoints. Blinding a notification costs 23.47 ms and a subscription 58.97 ms — $1848\times$ and $4643\times$ a broker-side Match (Figure 4). Both are one-off, off the routing path: a publisher blinds once per announcement, a subscriber once per subscription, and the result is reused for every subsequent match. This is the workload partition DNS++ intends, and it holds on real hardware.

Caveat on absolute latency. These absolute values are not throughput figures. Our harness dispatches all publications before polling subscriber sockets, and on two cores the inter-publication sleep overshoots substantially: the same configuration in a different driver run yielded a mean an order of magnitude higher. The comparison remains sound because the runs are paired — identical seeds and placement, executed back to back — so scheduling noise is common to both arms and cancels in the paired difference. We therefore report the comparison, not the absolute values.

5.6 Dynamics: Convergence After Replica Migration

The central hypothesis holds that DNS++ propagates service changes in seconds rather than the minutes-to-hours of TTL expiry. With 20 publishers and 50 subscribers on one service, we let the system settle, then perform 20 migrations per trial: move a random publisher to a new random location and re-publish, recording for every subscriber whose correct nearest replica changes the time until it receives that replica,

Table 6: Replica migration convergence, brake-free (20 publishers, 50 subscribers, 20 migrations per trial, 30 trials).

Migration class	n	Converged	Median (ms)	p95 (ms)
closer	1036	998 (96.3%)	0.42	1.24
farther	1391	0 (0.0%)	—	—

with a 1s timeout. All runs are brake-free, verified by the **braked** counter reading zero — a throttled publication would otherwise be indistinguishable from a failure to converge. Migrations fall into two classes under the monotone filter: **closer**, where the migrated replica lands nearer than the subscriber’s cached distance and is admitted; and **farther**, where the subscriber’s own replica moved away and its optimum is now a *different* replica.

Closer-class migrations converge in a median of 0.42ms, 95th percentile 1.24ms — against a typical 300s TTL, roughly five orders of magnitude faster, and comfortably inside the “seconds, not minutes” target. The 3.7% that did not converge are a second-order effect of the same filter: a subscriber whose cached distance was set by a replica that has since moved away retains that stale minimum, against which even a genuinely closer replica can be rejected.

Farther-class migrations never converge: zero of 1391. This is not an implementation defect but a consequence of the design. Updates are push-only and gated on strict improvement, so for a subscriber to learn of a better alternative after its replica moves away, that alternative must both re-publish *and* be strictly closer than a now-stale cached distance. Neither is guaranteed, and neither occurred. The dynamics claim must therefore carry a direction: DNS++ converges sub-millisecond when a service moves *closer* or a new replica appears, and not at all when one moves *away* or disappears, leaving affected subscribers with stale state until a closer publication happens to arrive or the subscription expires.

6 Discussion

6.1 The Brake Trade-off, and Why the Quadrant Cache Wins

Our measurements replace an inference with an observation: at **brake_limit=1** the single broker suppressed 206 of 330 publications and recall fell to 0.452, and the relationship is not one-to-one — recall exceeds $1 - \text{suppression}$ because suppression costs recall only when the suppressed publication was someone’s optimum. In the three-broker tree the trade is markedly worse: 6.9% of forwarding events saved for 77% of correct resolutions lost. The brake is therefore best understood not as an efficiency knob but as an overload valve — a way to keep a broker functioning under publication storms, at a recall cost severe enough that it should not engage in normal operation. It follows that any sensible setting must scale with publisher density: in our 20-publisher tree over four quadrants, a limit of 4 already suppresses a quarter of publications, so a deployment with thousands of replicas per quadrant would need a proportionally higher limit. An absolute count is the wrong unit; a fraction of observed quadrant load would degrade more gracefully.

The deeper point is selectivity. DNS++ contains two traffic-reduction mechanisms, and they are not equivalent. The quadrant cache is *information-aware*: it suppresses a publication precisely when a better one has already been forwarded for that region, so suppression is nearly free in recall terms — alone it reaches a traffic ratio of 1.168 at 0.981 recall, close to the floor a tree topology permits. The brake is *rate-based*: it counts events in a window and drops the excess, with no notion of whether what it drops was anyone’s optimum. The first is a filter; the second is a fuse. In a two-level tree the cache suffices alone and the brake should sit high enough never to engage; its value is as a safety mechanism, and potentially in deeper hierarchies where upward traffic accumulates — a regime our topology cannot exercise. An information-aware *upward* filter, applying quadrant-cache logic to the parent direction, would likely dominate the brake on both axes.

6.2 Privacy, Performance, and What the Measurement Establishes

The privacy layer’s cost is dominated by endpoint blinding, not broker routing: once blinding is amortised, the broker adds at most $S \times 0.0127$ ms per publication — under 0.635ms even on the pessimistic assumption that grouping fails entirely at the scale measured. This is the intended DNS++ workload partition, and it

holds on real hardware: expensive exponentiations sit with publishers and subscribers, which perform them infrequently and independently, while the routing infrastructure stays fast.

The qualification is that, as implemented, the broker also sees the plaintext service name. The result therefore establishes that homomorphic matching *can* be made cheap on a name-resolution critical path; it does not by itself establish that this implementation provides privacy. For latency-critical deployments, blinding could be reduced by smaller keys (trading security), GPU acceleration — the original paper reports 9.6M matches per second on an H100 — or caching blinded values for repeated names, none of which requires broker changes.

6.3 Real-System Effects Absent from Simulation

Three effects appeared that a deterministic simulator cannot produce. Kernel receive-buffer overflow set the practical ceiling on measurable scale, degrading recall while leaving stretch untouched. Trial-to-trial variance, absent by construction from sequential simulation, is large enough that a five-trial evaluation — as in our own earlier revision — yields intervals wide enough to support opposite conclusions. And scheduling jitter on two cores inflated absolute latency by an order of magnitude between runs of the same configuration, which is why we report only paired comparisons. None are design flaws in DNS++; identifying them is part of what an implementation study contributes.

6.4 Limitations

Privacy as implemented. Our brokers index routing tables by a hash of the plaintext service name rather than by ciphertext, because blinded values are semantically secure and cannot serve as lookup keys without destroying grouping (Section 3.4). The broker therefore learns the name, and the homomorphic Match is a formal re-check rather than the routing mechanism — *this is the configuration under which Section 5.5’s figures were obtained*. The hash is neither keyed nor collision-resistant: at a DNS-scale space of $\approx 2^{30}$ names its 64-bit output gives a collision probability near 3%, and a collision causes distinct names to match. Separately, the prototype co-locates HEPS with the root broker, which retains λ , e_m , d_m and r_m ; a party holding these can blind candidate names and match them against stored subscriptions, mounting a dictionary attack over a space whose entropy is far below 2048 bits. And the recovered magnitude leaks ordering information to any holder of r_m . We therefore make no claim of deployment-grade privacy against the broker. Decoupling routing keys from ciphertext via the Cover protocol (Nabeel et al., 2012), and separating HEPS from the broker, are the principled fixes.

One-directional convergence. The monotone filter gives sub-millisecond convergence when replicas move closer and none at all when a subscriber’s replica moves away (Section 5.6). Closing this requires soft state or an explicit invalidation path, neither present in the current design.

Measurement methodology. Our harness dispatches all publications for a trial before polling sockets, and the inter-publication sleep overshoots on two cores: the same configuration differed by an order of magnitude in absolute latency between driver runs, so we report latency only for the paired comparison. Beyond 200 subscribers, recall degrades because a single-threaded client cannot drain hundreds of sockets fast enough and the kernel drops datagrams; stretch does not degrade there, confirming routing remains correct for delivered publications. Both ceilings are in the harness, not the broker.

Implementation shortcuts. The equirectangular distance does not wrap at the antimeridian, so pairs spanning $\pm 180^\circ$ appear nearly a full circumference apart, and uniform lat/lon sampling over-samples the poles where the approximation is weakest; because ground truth and broker use the *same* function, internal consistency of recall and stretch is unaffected, but fidelity to geodesic distance is. Exponentiations with secret exponents use `mpz_powm` rather than the constant-time variant, leaving a timing side channel. The protocol carries no authenticator, so any host reaching a broker can forge HELLO, REGION_UPDATE or PUBLISH and corrupt MBH aggregation.

Scope. All processes run on loopback with no emulated delay or loss, so latency figures isolate protocol and CPU cost and should not be read as wide-area performance. The multi-broker evaluation uses a three-broker, two-level tree; deeper hierarchies are unmeasured, and the brake is expected to matter more there. Placement is uniformly random rather than population-weighted, limiting comparability with the original

simulation, and the evaluated scale remains orders of magnitude below its 10^6 endpoints. Finally, although our hypothesis is framed relative to conventional DNS, we do not measure a production resolver under matched conditions; the brake-unbounded configuration is an internal baseline only.

7 Conclusion

We have presented the first real-system implementation of the DNS++ broker architecture — Algorithm 1, a multi-broker overlay with dynamic MBH aggregation, and the modified Paillier privacy layer — together with a reproducible measurement harness.

Three results emerge. The propagation brake is blunter than simulation suggested: in a three-broker tree it buys a 6.9% reduction in forwarding events at the cost of 77% of correct resolutions, whereas the quadrant cache alone attains a traffic ratio of 1.168 at 0.981 recall. The distinction is selectivity — the cache suppresses what is redundant, the brake what exceeds a rate — and it argues for treating the brake as an overload valve rather than an efficiency parameter. The privacy layer’s broker-side cost is bounded by $S \times 0.0127$ ms in the number of subscription groups, three orders of magnitude below endpoint blinding, and a paired 30-trial comparison detects no latency difference ($p = 0.299$), validating the DNS++ workload partition on real hardware. And convergence after migration is directional: sub-millisecond when a replica moves closer, absent when one moves away.

We are explicit that the implementation does not deliver the privacy the design intends: restoring subscription grouping under semantically secure ciphertexts required indexing on a hash of the plaintext name, so the broker learns what it was meant not to. What we establish is that the homomorphic machinery is cheap enough to sit on a name-resolution critical path — a necessary condition for the design, not a sufficient one. Future work follows directly: an information-aware upward filter to replace the rate-based brake; the Cover protocol to recover the intended privacy property; soft state to make convergence bidirectional; a HEPS separated from the broker; and a harness able to exercise deeper hierarchies and larger endpoint counts.

References

- Radhakrishna Bhat and N. R. Sunitha. A novel private information retrieval technique for private DNS resolution. In *Proc. Intl. Conf. on Computational Intelligence and Networks*, 2019.
- Nikita Borisov, George Danezis, and Ian Goldberg. DP5: A private presence service. *Proceedings on Privacy Enhancing Technologies*, 2015.
- Matt Calder, Ashley Flavel, Ethan Katz-Bassett, Ratul Mahajan, and Jitendra Padhye. Analyzing the performance of an anycast CDN. In *Proceedings of the 2015 Internet Measurement Conference (IMC)*, 2015.
- Miguel Castro, Peter Druschel, Anne-Marie Kermarrec, and Antony I. T. Rowstron. SCRIBE: A large-scale and decentralized application-level multicast infrastructure. *IEEE Journal on Selected Areas in Communications*, 20(8), 2002.
- D. W. Chadwick. Understanding X.500 - the directory, 1994.
- Benny Chor, Oded Goldreich, Eyal Kushilevitz, and Madhu Sudan. Private information retrieval. *Journal of the ACM*, 45(6), 1998.
- C. Contavalli, W. van der Gaast, D. Lawrence, and W. Kumari. RFC 7871: Client subnet in DNS queries. RFC 7871, Internet Engineering Task Force, 2016. URL <https://www.rfc-editor.org/rfc/rfc7871>.
- Abhishek Gupta, Ozgur D. Sahin, Divyakant Agrawal, and Amr El Abbadi. Meghdoot: Content-based publish/subscribe over P2P networks. In *Proceedings of the 5th ACM/IFIP/USENIX International Conference on Middleware*, 2004.
- P. Hoffman and P. McManus. RFC 8484: DNS queries over HTTPS (DoH), 2018.
- Z. Hu and Z. Zhu. RFC 7858: Specification for DNS over transport layer security (TLS), 2016.

-
- Ken Huang, Vineeth Sai Narajala, Idan Habler, and Akram Sheriff. Agent name service (ANS): A universal directory for secure AI agent discovery and interoperability, 2025.
- C. Huitema, S. Dickinson, and A. Mankin. RFC 9250: DNS over dedicated QUIC connections, 2022.
- Karel Hynek. *The impact of encrypted DNS on network security*. PhD thesis, PhD thesis, 2023.
- Ryan Kurte, Zoran Salcic, and Kevin I-Kai Wang. Decentralised global service discovery for the internet of things. *Sensors*, 24(7), 2024.
- P. Mockapetris. RFC 1034: Domain names - concepts and facilities, 1987.
- Mohamed Nabeel, Ning Shang, and Elisa Bertino. Efficient privacy preserving content based publish subscribe systems. In *Proceedings of ACM Symposium on Access Control Models and Technologies (SACMAT)*, 2012.
- Mohamed Nabeel, Stefan Appel, Elisa Bertino, and Alejandro Buchmann. Privacy preserving context aware publish subscribe systems. In *Network and System Security (NSS)*, 2013.
- Pavlos Papadopoulos, Nikolaos Pitropakis, William J Buchanan, Owen Lo, and Sokratis Katsikas. Privacy-preserving passive DNS. *Computers*, 9(3), 2020.
- Sylvia Ratnasamy, Paul Francis, Mark Handley, Richard Karp, and Scott Shenker. A scalable content-addressable network. In *Proceedings of the 2001 Conference on Applications, Technologies, Architectures, and Protocols for Computer Communications*, SIGCOMM '01, 2001.
- Miguel Rio et al. Beyond DNS: A privacy-first, dynamic, location-aware name resolution system. In *SIGCOMM '26, Paper #591*, 2026.
- Paul Schmitt, Anne Edmundson, Allison Mankin, and Nick Feamster. Oblivious DNS: Practical privacy for DNS queries. In *Proceedings of the 2019 Applied Networking Research Workshop*, 2019.
- J. Sermersheim. RFC 4511: Lightweight directory access protocol (LDAP): The protocol, 2006.
- T. Wicinski. RFC 9076: DNS privacy considerations, 2021.

A TLV Protocol Specification

The fixed 8-byte header carries `msg_type` (uint16), `num_tlvs` (uint16) and `payload_len` (uint32).

Table 7: TLV field type definitions.

Type name	Code	Size
COORDINATES	0x0001	8 bytes (float lat, float lon)
FLAGS	0x0002	4 bytes (uint32_t bitmask)
SERVICE_NAME	0x0003	Variable (UTF-8 string)
BRAKE_LIMIT	0x0004	4 bytes (int32_t)
REGION	0x0005	16 bytes (4× float MBH)
STATS_DATA	0x0006	32 bytes (4× uint64_t)
STATS_DATA_EXT	0x0007	96 bytes (12× uint64_t)
BLINDED_VALUE	0x0010	Variable (hex string)
BLINDED_VALUE_HI	0x0011	Variable (hex string)

`STATS_DATA_EXT` carries, in order: upward forwards, downward forwards, local deliveries, total braked, Match invocations, Match hits, publications received, subscriptions received, subscription groups, homomorphic mode flag, upward braked, locally braked. `STATS_DATA` is retained for backward compatibility.

B Broker Configuration and Harness Usage

```
# leaf1.conf (London); root.conf omits parent_addr
broker_id=broker_leaf_1
listen_port=9001
parent_addr=127.0.0.1:9000
coords=51.5,-0.1
brake_limit=4
brake_window=10
brake_scope=both      # upward | local | both
require_he=false      # true: exit if the HEPS key is missing
```

The broker accepts only a configuration file path; an unreadable file, unknown key or out-of-range value causes it to exit rather than fall back to defaults. All results were produced by a single driver that starts a fresh broker per configuration, skips existing output so an interrupted run resumes, and records environment metadata alongside the data.

```
bash scripts/run_full_eval.sh --trials=30 --out=results/final

./bench_broker <ip> <port> <num_pubs> <num_subs> <brake_limit> \
               <num_trials> <seed> <encrypted> --warmup=3

./bench_dynamics <ip> <port> <num_pubs> <num_subs> \
                <num_migrations> <trials> <seed> <encrypted> \
                --timeout-ms=1000          # must be run brake-free
```

Per-subscriber records go to stdout as CSV carrying seed, trial index and derived per-trial seed on every row; aggregate summaries, environment metadata and kernel UDP error deltas go to stderr.

C Cryptographic Parameters and Micro-Benchmark

The modified Paillier implementation uses a 2048-bit modulus n — verified stable, 20 of 20 key generations producing exactly 2048 bits after forcing the top two bits of each prime factor — with $l = 64$ (hash width), $s = 2047$, $u = 1024$, a 992-bit stretching factor r_m , and per-execution random $0 < r < r_m$. Corollary 1

requires $r_m|x-v|_{\max} + r < n/2$; with $|x-v| < 2^{64}$ and $r_m < 2^{992}$ the left side is bounded by $2^{1056} + 2^{992} \ll 2^{2046}$, a margin of roughly 2^{990} .

Table 8: Cryptographic operations (2048-bit Paillier; median of three runs of 1000 iterations).

Operation	Executed by	Median (ms/op)	Range
blindNotification	Publisher	23.474	[20.03, 24.22]
blindSubscription	Subscriber	58.967	[54.69, 65.61]
executeMatch	Broker	0.0127	[0.012, 0.015]

Broker-side matching is $1848\times$ cheaper than blinding a notification and $4643\times$ cheaper than blinding a subscription.

D Supplementary Figures

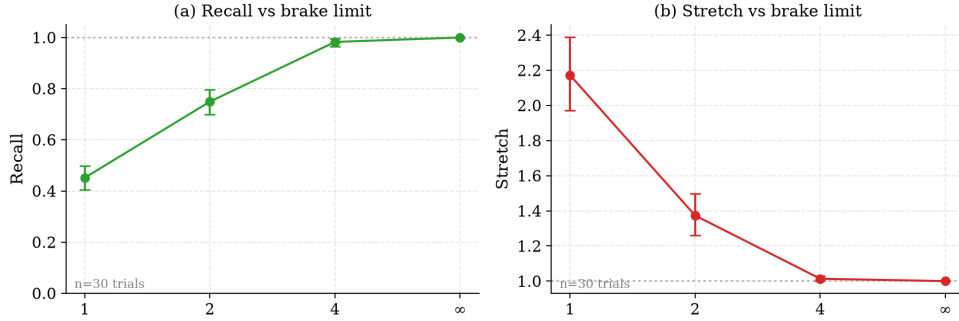


Figure 5: Single-broker brake sweep: recall and stretch against brake limit (∞ denotes `brake_limit=1000`), 95% bootstrap CIs over 30 trials (data in Table 2).

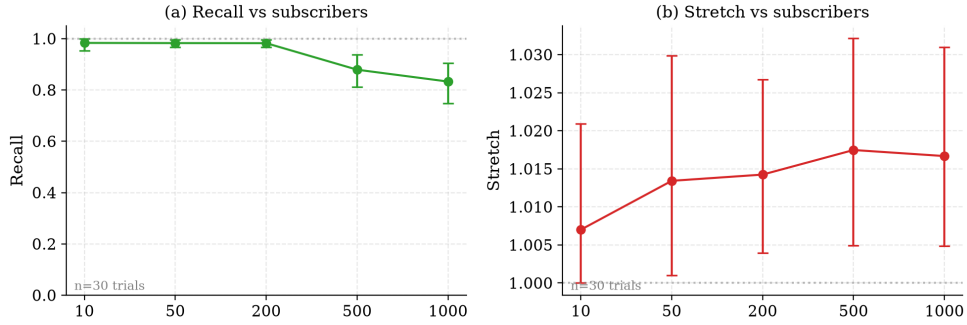


Figure 6: Subscriber-count scaling: recall and stretch, 95% bootstrap CIs over 30 trials (data in Table 4).

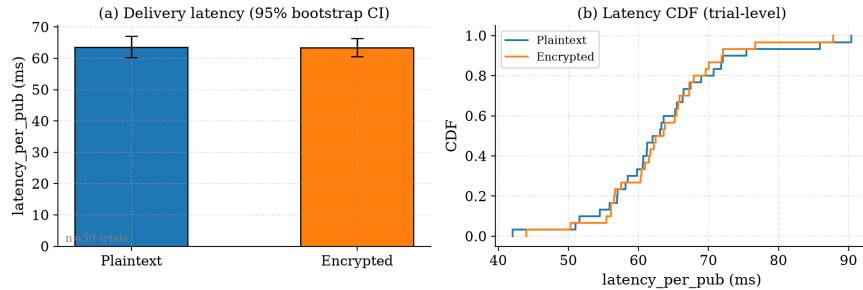


Figure 7: Plaintext versus encrypted per-publication latency, 95% bootstrap CIs over 30 paired trials (data in Table 5).

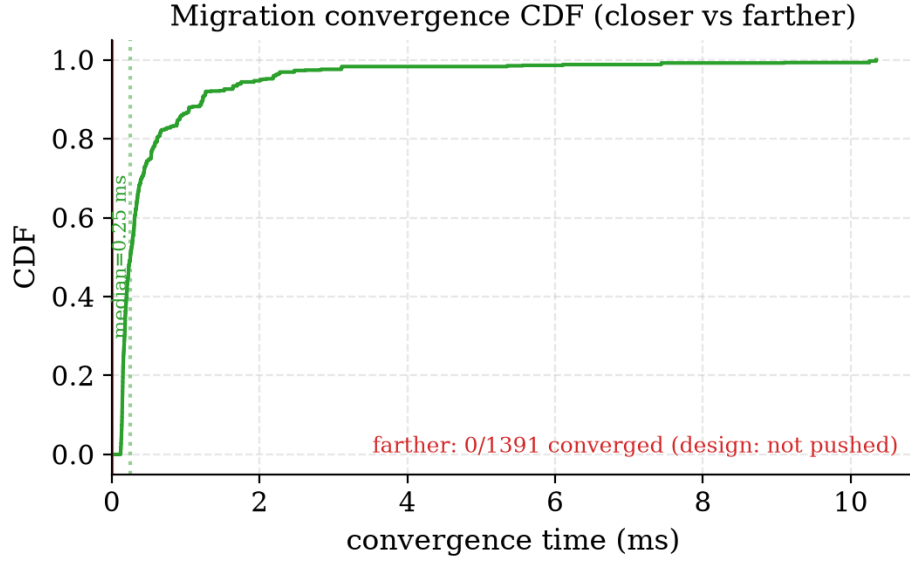


Figure 8: Cumulative distribution of convergence time for closer-class migrations. Farther-class migrations do not converge and are absent from the distribution.

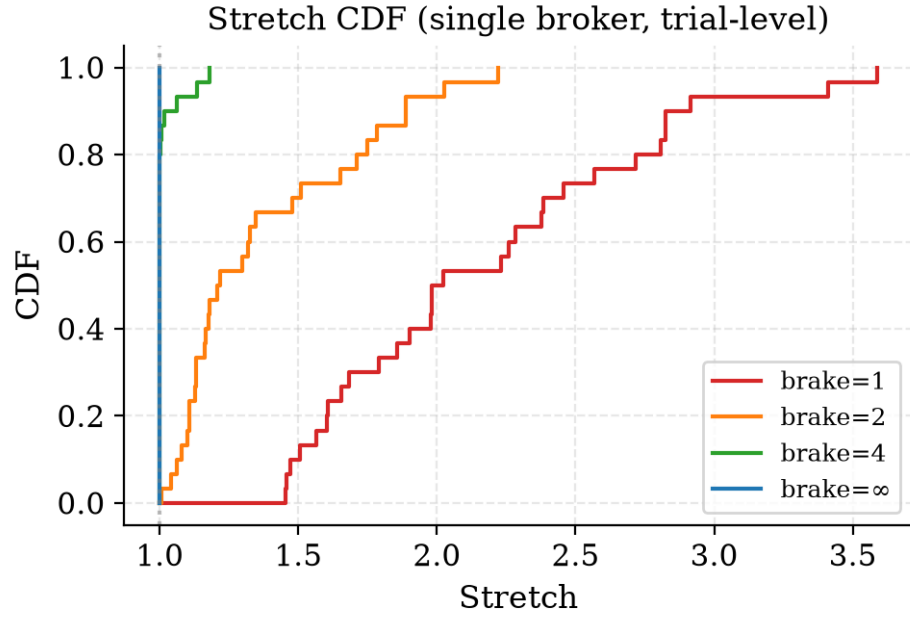


Figure 9: Cumulative distribution of stretch by brake limit (∞ denotes `brake_limit=1000`), 30 trials. Tightening the brake leaves most subscribers at optimum while displacing a minority substantially.